

CS630 Graduate Algorithms

November 1, 2022

by Dora Erdos

- Randomized algorithms
- QuickSort - very high level sketch KT ch. 13.5
- Contention Resolution KT ch. 13.1

Why randomized algorithms?

sublinear algorithms

deterministic algorithms to solve problems:

- find an exact (or optimal) solution
- worst-case running time

approximation algorithms:

- solution is acceptable close to optimal
- worst case running time is better than for the optimal algorithm
 - but may still be infeasible

Set Cover greedy:
each iter:
• select set with largest additional gain
• update the gain of all other sets

$O(nm)$

greedy: $O(k \cdot \ln n)$ sets

↓
function of n !

randomization:

- deterministic algorithms on random input
 - average-case running time - randomized QuickSort
- algorithms that behave random
 - they are “good” if they succeed with some probability
 - “succeed” - either in terms of runtime or quality of solution

probability exercises



A family has two children.

- What is the probability that both are girls?

$$\frac{1}{4}$$

- When asked, the father says that his older child is a girl. Given this information, what is the probability of having two daughters?

$$\frac{1}{2}$$

- When asked, the father says that he has a daughter. Given this information, what is the probability of having two girls?

$$\frac{1}{3}$$

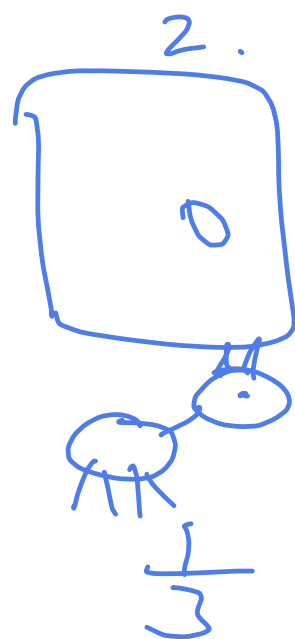
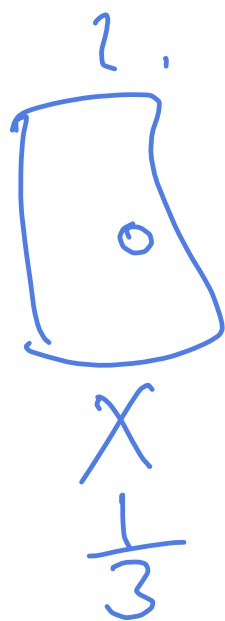
probability exercises

Monty Hall problem:

(Monty Hall was the host of the program Let's Make a Deal)

There are 3 doors, behind one of the doors is a car, the other two doors hide goats.

- The contestant selects one of the doors
- The host then opens one of the remaining two doors that hides a goat
- Does it matter whether the contestant switches doors if they want to win the car?



switch 1
car goat goat
goat goat car
X goat car goat

probability conditioned on not door 2.

$\frac{1}{2}$? $\frac{2}{3}$?

Conditional probabilities

Let A and B be events.

conditional probability:

$$P(A|B) = \frac{P(A \cap B)}{P(B)}$$

Bayes' theorem:

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

Chain rule:

$$P(A_n \cap A_{n-1} \cap \dots \cap A_1) = \prod_{k=1}^n P\left(A_k \mid \bigcap_{j=1}^{k-1} A_j\right)$$

$$P(A_1 \cap A_2 \cap A_3 \cap A_4) = P(A_4|A_3 \cap A_2 \cap A_1) \cdot P(A_3|A_2 \cap A_1) \cdot P(A_2|A_1) \cdot P(A_1)$$

Event space: all possible outcomes

events: A, B are of the outcomes

want to know
↓

often this is
↓ easier to compute

$$P(A|B)P(B) = P(B|A)P(A)$$

using randomness to get definitive results - example

2 biased coins, head is 60% likely. How can we use this to simulate an unbiased coin?

2 flips

HH $\rightarrow$ repeat both flips
HT $\rightarrow$ same prob. $\leftarrow$ "heads" 0.6×0.4
TH $\rightarrow$ same prob. $\leftarrow$ "tails" 0.4×0.6
TT $\rightarrow$ repeat both

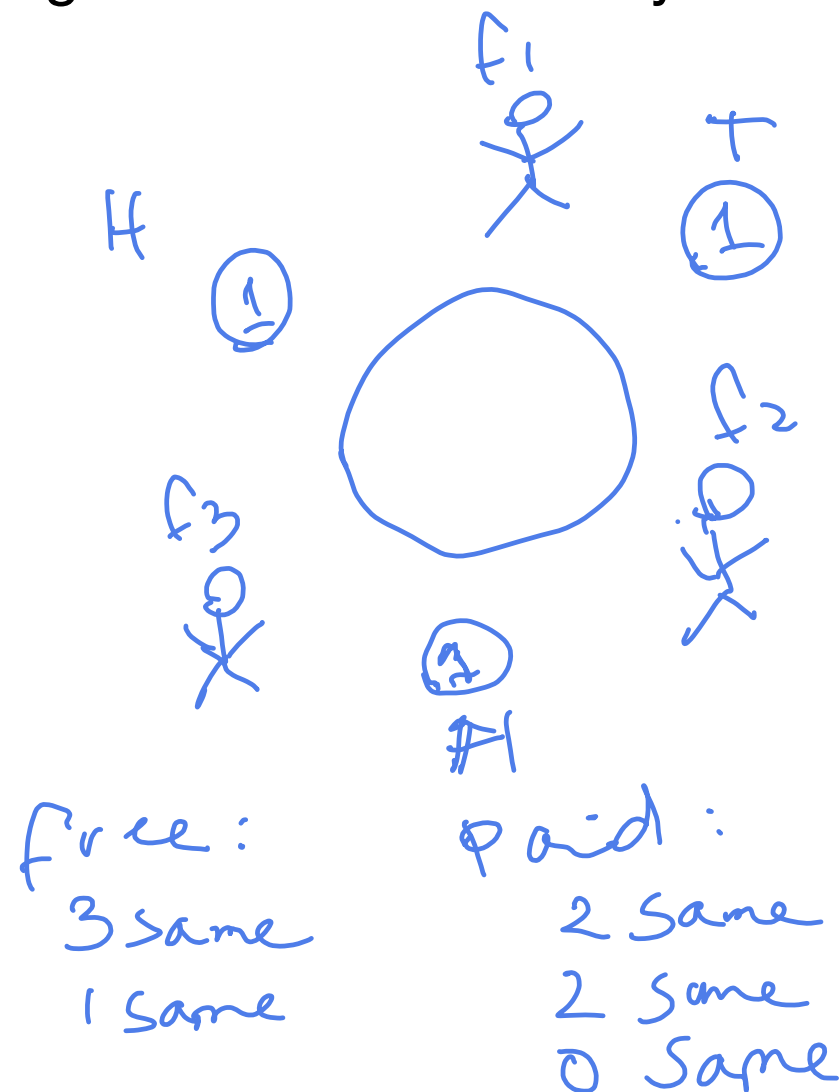
general prob. p .

4 people? using randomness to get definitive results - example 2

Cryptography
Privacy

3 friends are sitting at a round table at the restaurant.

- either one of them pays the bill or they get a free meal from the restaurant
- they want to find out whether they got the free meal
- they agree, that if it was one of them paying, they don't want to reveal that
- how can they decide definitively whether the meal was free, while keeping the generous friend anonymous?



the coins I saw were the same
different

if a friend didn't pay:
report truth

if paid: lie

count the 'same' responses

• n people
• coin flips between each pair:
would this work?

using randomness to get definitive results - example 2

3 friends are sitting at a round table at the restaurant.

- either one of them pays the bill or they get a free meal from the restaurant
 - they want to find out whether they got the free meal
 - they agree, that if it was one of them paying, they don't want to reveal that
 - how can they decide definitively whether the meal was free, while keeping the generous friend anonymous?
-
- Algorithm:
 - each person flips a coin between them and the person on their right
 - only they can see the coin
 - each person reports whether the coins on their two sides are the same or not
 - the person who paid will state the opposite of the truth, the other answer truthfully
 - claim: if the reported number of equal coins is odd, then everybody answered truthfully, hence the meal was free

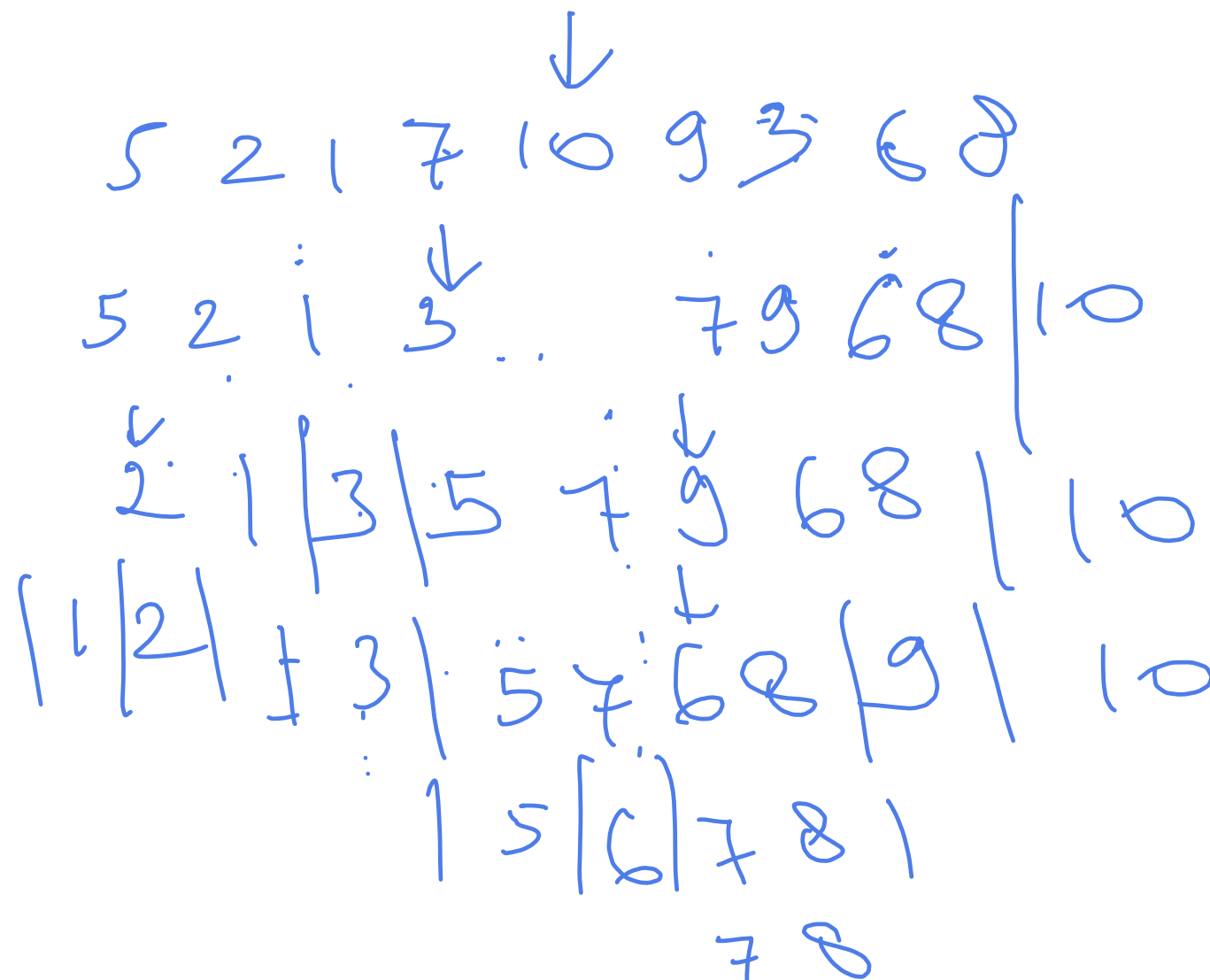
using randomness to get definitive results - example 3

Quicksort (input array A):

- Recurse on subarrays until sorted
 - select middle element as pivot
 - assign values less than pivot to left, greater to right subarray

$O(n^2)$
if [↑] the middle index value

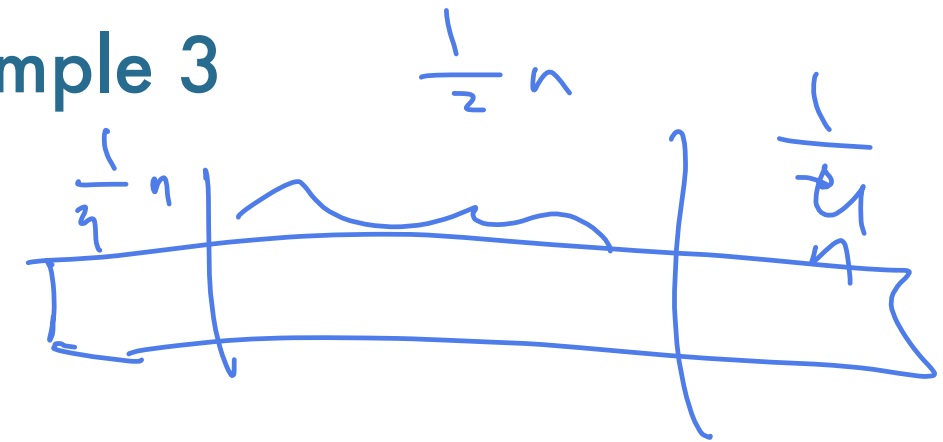
Worst case running time?



is on one end of the range, then the length of subarray is reduced by 1.

$$c \cdot n + c(n-1) + c(n-2) + \dots = c \sum_{i=1}^n i = O(n^2)$$

using randomness to get definitive results - example 3



Randomized QuickSort:

- Recurse on subarrays until sorted
 - select random element as pivot
 - assign values less than pivot to left, greater to right subarray

expected number of iterations i.e. expected running time $O(n \log n)$ is better than worst case $O(n^2)$

- proof is based on computing for each value how many times (in expectation) it will be compared to a pivot
- turns out that if the pivot is in the middle half (between $n/4$ and $3n/4$) of the number range, then we get the favorable runtime
- what does expected running time mean?
- how to use it in practice?
 - if the number of iterations $c \cdot n \log n$ is not sorted by this time $\rightarrow$ start again from scratch

Random running time vs. random result

Suppose that we have a binary array of length n with equal number of 0s and 1s.

Find an index of any 1

Las Vegas method: pick an index at random and check its value. If 1, then return the index, otherwise repeat.

*randomized
Quick Sort*

- + guaranteed to find one
- expected number of trials might be large

Monte Carlo method: fix the number of iterations to k . For at most k tries check a random index, return if 1.

- + finds a 1 with some probability
- deterministic running time

Both:
"success" happens with some probability
LV: limited number of iterations
MC: we find an index

Contention resolution

n processes $P_1, P_2, \dots, P_n$ are competing for access to a single shared database

- time is divided into discrete rounds
- at most one process can access the database in a single round
- if more than one processes try to access it, they are all locked out for the round

How can we divide the rounds so that all processes can get through on a regular basis?

- Processes agree on some order
 - assumes that P_i s can communicate.
- The processes cannot communicate
 - in each round each process makes a request with probability p .
 - what is a good value for p ? $\rightarrow \frac{1}{n}$
 - $\rightarrow$ the decisions of each process are independent

Contention resolution

n processes $P_1, P_2, \dots, P_n$ are competing for access to a single shared database

- time is divided into discrete rounds
- at most one process can access the database in a single round
- if more than one processes try to access it, they are all locked out for the round

How can we divide the rounds so that all processes can get through on a regular basis?

- if the processes can communicate — agree on some order of access
- suppose they can't:
 - in each round each process makes an attempt with probability p
 - what is a good value for p ?
 - how good is this strategy?

Measuring performance

In each round each process makes an attempt with probability p

How should we measure the performance of this algorithm?

Success:

- minimize number of rounds where there is a clash
- minimize # rounds until everybody has access



our choice is what the value of p is.

objective function:

the expected # of rounds

Measuring performance

In each round each process makes an attempt with probability p

$A[i,t]$ = the event that process P_i makes an attempt at time t

$S[i,t]$ = the event that P_i makes an attempt at time t and succeeds

probability of success (given n processes):

$$P(S[i,t]) = P(1-p)^{n-1}$$

P_i making an attempt

$n-1$

other processes do not.

are these probabilities independent?

yes.

What value of p can maximize this probability?

if $p=1$
 $p=0$
is pos.

$$P(S[i,t]) = 0$$

$$\left(p(1-p)^{n-1} \right)' = (1-p)^{n-1} - \therefore (n-1)p(1-p)^{n-2}$$

Measuring performance

In each round each process makes an attempt with probability p

$A[i,t]$ = the event that process P_i makes an attempt at time t

$S[i,t]$ = the event that P_i makes an attempt at time t and succeeds

probability of success (given n processes):

$$P(S[i, t]) = p(1 - p)^{n-1}$$

are these probabilities independent?

- yes, that is how the algorithm is designed

What value of p can maximize this probability?

$$p(1 - p)^{n-1}$$

- nonnegative function.
- take the derivative to find its max
- It's maximum is at $p=1/n$

Expected success

calculus $\left(1 - \frac{1}{t}\right)^t \rightarrow \frac{1}{e}$

probability of P_i succeeding at time t : $P(S[i, t]) = p(1 - p)^{n-1} = \frac{1}{n} \left(1 - \frac{1}{n}\right)^{n-1}$

for $p=1/n$

- from $1/4$ up to $1/e$ $\lim_{n \rightarrow \infty} \frac{1}{n} \left(1 - \frac{1}{n}\right)^n = \frac{1}{e}$
- from $1/2$ down to $1/e$ $\lim_{n \rightarrow \infty} \frac{1}{n} \left(1 - \frac{1}{n}\right)^{n-1} = \frac{1}{e}$

We get $\frac{1}{en} \leq P(S[i, t]) \leq \frac{1}{2n}$ hence $P(S[i, t]) = \Theta\left(\frac{1}{n}\right)$

in the book

low numbers,
but > 0

Expected success

probability of P_i succeeding at time t with $p=1/n$: $P(S[i, t]) = \frac{1}{n} \left(1 - \frac{1}{n}\right)^{n-1}$

we have: $P(S[i, t]) = \Theta\left(\frac{1}{n}\right)$ ← for each round.

How long do we have to wait for success?

$F([i, t])$ = the event that P_i doesn't succeed after t rounds

meaning:
decide in advance
how many attempts

The probability that we don't succeed

- after $O(n)$ rounds

$$P(F[i, t]) \leq \frac{1}{e} \approx 0.63$$

- after $O(n \log(n))$ rounds

$$P(F[i, t]) \leq n^{-c}$$

(proof in KT book)

↑ depending on what the const
is in $O()$ we can set 'c'.

Expected success

probability of P_i succeeding at time t with $p=1/n$: $P(S[i, t]) = \frac{1}{n} \left(1 - \frac{1}{n}\right)^{n-1} = \Theta\left(\frac{1}{n}\right)$

How many rounds before all processes get through?

$F([i, t])$ = the event that P_i doesn't succeed after t rounds

probability that t rounds are not enough: $P\left(\cup_{i=1}^n F[i, t]\right)$

Union Bound: given events $E_1, E_2, \dots, E_n$ we have

$$P\left(\cup_{i=1}^n E_i\right) \leq \sum_{i=1}^n P(E_i)$$

We know $P(F[i, t]) \leq n^{-c}$

so by the union bound $P\left(\cup_{i=1}^n F[i, t]\right) \leq \sum_{i=1}^n P(F[i, t]) \leq n \cdot n^{-c}$

If we select $t = 2 \lceil en \rceil \ln n$ then this result implies we succeed with prob at least $1 - n^{-1}$

Sufficient long time

Success very likely